

Проект по предметот Бази на Податоци

Дизајн и имплементација на систем за управување со резервации и деловни договори во заеднички работни центри (Coworking Spaces) - CoWorkHub

Мартина Петковска, 223313

Факултет за информатички науки и компјутерско инженерство, Скопје
Ментор: д-р Горан Велинов

Апстракт: Овој проект го презентира CoWorkHub, систем кој овозможува флексибилно изнајмување на работен простор преку поддршка на еднократни резервации и долгорочни деловни договори. Системот користи напреден модел на податоци за динамичко конфигурирање на просторот и автоматизирано генерирање на фактури.

Клучни зборови: Coworking, Резервации, Деловни договори, Фактури.

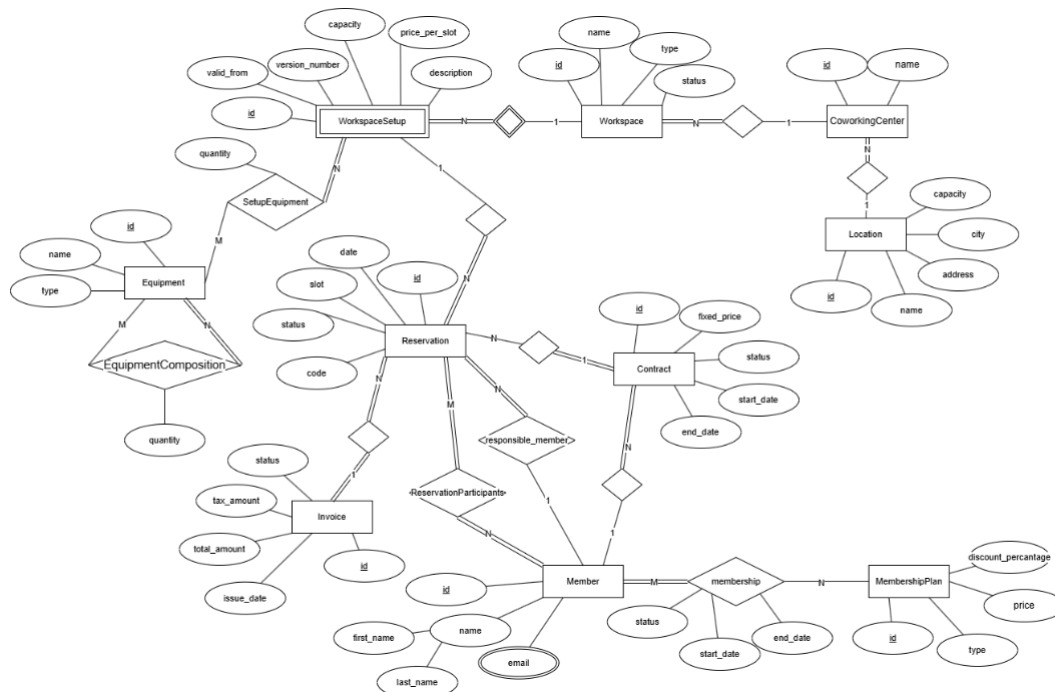
1 Опис на функционалностите на системот

CoWorkHub претставува систем за управување со резервации и деловни договори во Coworking центри. Системот поддржува два модели на користење: еднократни резервации за индивидуални корисници и долгорочен закуп преку деловни договори.

Системот овозможува управување со центри, нивните локации и работни простори. За секој центар се чува неговото име, додека локациите содржат детални податоци за адресата. Секој центар располага со повеќе работни простори (Workspace), кои имаат дефиниран тип, капацитет и статус. Работните простори можат да имаат различни конфигурации (WorkspaceSetup) низ времето, кои содржат специфична опрема и нејзини под-компоненти.

Членовите можат да користат различни временски претплати (MembershipPlan) и да вршат еднократни резервации (Reservation) за конкретен сетоп или закуп преку договори (Contract). Во рамки на резервација, системот автоматски генерира фактура (Invoice). Додека, во рамките на договорите, системот поддржува закуп на повеќе работни единици со дефиниран попуст и времетраење, како и автоматско генерирање месечни фактури (Invoice) за сите активности поврзани со договорот.

2 Опис на моделот на податоци



2.1 Опис на моделот на податоци

Релациониот модел на податоци е систематски изведен од концептуалниот ЕР-дијаграм со примена на релационите правила за мапирање, нормализација и зачувување на референцијалниот интегритет.

Со цел полесна организација, моделот на податоци на системот CoWorkHub е организиран во неколку логички целини: *управување со локации и центри, организација на работни простори, членство и трансакции.*

Ентитетите Location и CoworkingCenter овозможуваат евиденција на физички локации и центри, додека Workspace и WorkspaceSetup поддржуваат различни конфигурации на работни простори со специфична цена, капацитет и намена.

За управување со опрема се користат Equipment, SetupEquipment и EquipmentComposition, со што се овозможува поврзување на повеќе типови опрема и нивни составни делови.

Корисниците се моделирани преку Member, MemberEmail, MembershipPlan и Membership, што овозможува поддршка за повеќе претплатнички планови.

Трансакцискиот дел е имплементиран преку Reservation, ReservationParticipants, Invoice и Contract, при што системот поддржува еднократни резервации и долгорочни деловни договори со автоматско фактурирање.

2.2 Релационен модел на податоци

Релациониот модел е дефиниран преку следните релации:

Location(id, name, address, city)

CoworkingCenter(id, name, location_id*)

Workspace(id, name, capacity, type, status, coworking_center_id*)

WorkspaceSetup(id, workspace_id*, version_number, valid_from, price_per_slot, description)

Equipment(id, name, type)

SetupEquipment(setup_id*, equipment_id*, quantity)

EquipmentComposition(parent_equipment_id*, child_equipment_id*, quantity)

Member(id, first_name, last_name)

MemberEmail(member_id*, email)

MembershipPlan(id, type, price, discount_percentage)

Membership(member_id*, plan_id*, start_date, end_date, status)

Reservation(id, date, slot, status, code, responsible_member_id*, setup_id*, invoice_id*, contract_id*)

ReservationParticipants(reservation_id*, member_id*)

Invoice (id, issue_date, total_amount, tax_amount, status)

Contract (id, fixed_price, status, start_date, end_date, member_id*)

3 SQL трансформација - Трансформација на ЕР-моделот во Релационен модел

Трансформацијата на концептуалниот ЕР-дијаграм во релациона база на податоци (PostgreSQL) е извршена според стандардните правила за мапирање.

3.1 Креирање на табели (DDL)

Системот CoWorkHub е имплементиран во PostgreSQL и користи релационен модел со 15 меѓусебно поврзани табели. Во базата се дефинирани примарни и надворешни клучеви, CHECK ограничувања, UNIQUE правила и индекси за оптимизација на пребарувања по датум и корисник.

3.1.1 Независни табели:

Прво се креираат табелите кои произлегуваат од силни ентитети без зависности:

- Location, Equipment, Member, MembershipPlan, Invoice.
- Во табелата **Invoice** се прават и персонализирани CHECK ограничувања:
 status VARCHAR(20) DEFAULT 'unpaid'
 CHECK (status IN ('paid', 'unpaid', 'refunded')),
 type VARCHAR(50)
 CHECK (type IN ('single', 'contract'))

Овие табели немаат Foreign Keys и можат да се креираат први. Секоја користи SERIAL PRIMARY KEY за автоматско генерирање на идентификатор.

3.1.2 Зависни табели:

По независните, се креираат табелите кои зависат од нив преку врски *еден-кон-многу* (т.е. во себе содржат надворешен клуч):

- **CoworkingCenter** – зависи од Location:
 location_id INT NOT NULL REFERENCES Location(id)
 ON DELETE RESTRICT ON UPDATE CASCADE
 - RESTRICT спречува бришење на локација додека постои центар во неа.
- **Workspace** – зависи од CoworkingCenter:
 coworking_center_id INT NOT NULL REFERENCES CoworkingCenter(id)
 ON DELETE CASCADE ON UPDATE CASCADE
 - CASCADE – при бришење на центар, се бришат и неговите работни простории
 - CHECK ограничувања за статус и капацитет
- **WorkspaceSetup** зависи од Workspace:
 workspace_id INT NOT NULL REFERENCES Workspace(id)
 ON DELETE CASCADE ON UPDATE CASCADE

UNIQUE (workspace_id, version_number)

- UNIQUE ограничувањето осигурува дека за ист простор не може да постојат две конфигурации со ист број на верзија.

- **Contact** зависи од Member:

member_id INT NOT NULL REFERENCES Member(id)
ON DELETE RESTRICT ON UPDATE CASCADE
CHECK (start_date < end_date)

- CHECK ограничувањето осигурува дека датумот на почеток е пред датумот на крај.

- **Reservation** – зависи од табелите Member, WorkspaceSetup, Invoice и Contract:

responsible_member_id INT NOT NULL REFERENCES Member(id),
setup_id INT NOT NULL REFERENCES WorkspaceSetup(id),
invoice_id INT NOT NULL REFERENCES Invoice(id),
contract_id INT NOT NULL REFERENCES Contract(id),
UNIQUE (setup_id, date, slot)

- invoice_id и contract_id се nullable — резервацијата може да постои пред да е генерирана фактура, и не мора да биде дел од договор.
- UNIQUE (setup_id, date, slot) спречува двојна резервација на ист простор во ист термин на ист датум.

3.1.3 M:N Врски и рекурзивни врски.

На крај се креираат табелите кои произлегуваат од врски многу-кон-многу и рекурзивни врски — бидејќи тие зависат од две или повеќе веќе-креирани табели:

- **EquipmentComposition** рекурзивна M:N врска на Equipment со самата себе:

PRIMARY KEY (parent_equipment_id, child_equipment_id)
FOREIGN KEY (parent_equipment_id) REFERENCES Equipment(id)
ON DELETE CASCADE,
FOREIGN KEY (child_equipment_id) REFERENCES Equipment(id)
ON DELETE CASCADE

- **ReservationParticipants** – M:N врска помеѓу Reservation и Member:

PRIMARY KEY (reservation_id, member_id),
FOREIGN KEY (reservation_id) REFERENCES Reservation(id)
ON DELETE CASCADE ON UPDATE CASCADE,
FOREIGN KEY (member_id) REFERENCES Member(id)
ON DELETE CASCADE ON UPDATE CASCADE

- При бришење на резервација или член, записите автоматски се отстрануваат.

- **Membership** поврзува Member со MembershipPlan со временски атрибути:

CHECK (start_date < end_date)
status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (status IN ('active', 'inactive')),

Приказ на дел од структурата:

```
CREATE TABLE Location ( id SERIAL ... );
CREATE TABLE Equipment ( id SERIAL ... );
CREATE TABLE Member ( id SERIAL ... );
CREATE TABLE MembershipPlan ( id SERIAL ... );
CREATE TABLE Invoice ( id SERIAL ... );
CREATE TABLE CoworkingCenter ( id SERIAL ... );
CREATE TABLE Workspace ( id SERIAL ... );
CREATE TABLE WorkspaceSetup ( id SERIAL ... );
CREATE TABLE Contract ( id SERIAL ... );
CREATE TABLE Reservation ( id SERIAL ... );
CREATE TABLE SetupEquipment ( setup_id ... );
CREATE TABLE EquipmentComposition ( parent_equipment_id ... );
CREATE TABLE ReservationParticipants ( reservation_id ... );
CREATE TABLE Membership ( id SERIAL ... );
CREATE TABLE MemberEmail ( member_id ... );

CREATE TABLE Reservation (
  id SERIAL PRIMARY KEY,
  date DATE NOT NULL,
  slot VARCHAR(20) NOT NULL
  CHECK (slot IN ('morning', 'afternoon', 'evening')),
  status VARCHAR(20) NOT NULL DEFAULT 'pending'
  CHECK (status IN ('pending', 'confirmed', 'cancelled')),
  code VARCHAR(20) NOT NULL UNIQUE,
  responsible_member_id INT NOT NULL,
  setup_id INT NOT NULL,
  invoice_id INT NULL,
  contract_id INT NULL,
  FOREIGN KEY (responsible_member_id) REFERENCES Member(id)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (setup_id) REFERENCES WorkspaceSetup(id)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (invoice_id) REFERENCES Invoice(id)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (contract_id) REFERENCES Contract(id)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  UNIQUE (setup_id, date, slot)
);

CREATE TABLE Invoice(
  id SERIAL PRIMARY KEY,
  issue_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  total_amount NUMERIC(10,2) NOT NULL
  CHECK (total_amount >= 0),
  tax_amount NUMERIC(10,2) NOT NULL,
  status VARCHAR(20) DEFAULT 'unpaid'
  CHECK (status IN ('paid', 'unpaid', 'refunded')),
  type VARCHAR(50)
  CHECK (type IN ('single', 'contract'))
);
```

3.2 Индекси

По креирањето на табелите, се додаваат индекси за оптимизација на најчестите пребарувања:

```
-- index for faster queries
CREATE INDEX idx_reservation_date ON Reservation (date);
CREATE INDEX idx_reservation_member ON Reservation(responsible_member_id);
CREATE INDEX idx_membership_member ON Membership(member_id);
```

Индексот по date го забрзува пребарувањето на резервации за конкретен ден (користено во извештајот за пополнетост). Индексот по responsible_member_id го оптимизира пребарувањето на резервации по член.

3.3 Процедури

Во системот се имплементирани складирани процедури (Stored Procedures) за автоматизација на дел од бизнис логиката поврзана со фактурирање и обработка на резервации. Процедурите се повикуваат од Django апликацијата преку signals при креирање и ажурирање на резервации.

Имплементирани се следниве процедури:

1. **generate_single_invoice** – генерира индивидуална фактура за резервација без договор

```
CREATE OR REPLACE PROCEDURE
generate_single_invoice(p_reservation_id INT,
p_setup_id INT, p_member_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    base_price NUMERIC(10,2);
    discount DECIMAL(5,2) DEFAULT 0;
    final_price NUMERIC(10,2);
    new_invoice_id INT;
BEGIN
    SELECT price_per_slot INTO base_price
    FROM WorkspaceSetup WHERE id = p_setup_id;

    SELECT COALESCE(mp.discount_percentage, 0) INTO
discount
    FROM Membership m
    LEFT JOIN MembershipPlan mp ON m.plan_id =
mp.id
    WHERE m.member_id = p_member_id
    AND m.status = 'active'
    AND CURRENT_DATE BETWEEN m.start_date AND
m.end_date
    LIMIT 1;

    final_price := base_price * (1 - discount / 100);

    INSERT INTO Invoice (total_amount, tax_amount,
status, type)
    VALUES (final_price, final_price * 0.18, 'unpaid',
'single')
    RETURNING id INTO new_invoice_id;

    UPDATE Reservation
    SET invoice_id = new_invoice_id
    WHERE id = p_reservation_id;
END;
$$;
```

2. **generate_contract_invoice** – групира договорни резервации и генерира заедничка фактура

```
CREATE OR REPLACE PROCEDURE
generate_contract_invoice(p_reservation_id INT,
p_setup_id INT, p_contract_id INT)
LANGUAGE plpgsql AS $$
DECLARE
slot_price NUMERIC(10,2);
existing_invoice_id INT;
BEGIN
SELECT price_per_slot INTO slot_price
FROM WorkspaceSetup WHERE id = p_setup_id;

SELECT invoice_id INTO existing_invoice_id
FROM Reservation
WHERE contract_id = p_contract_id
AND invoice_id IS NOT NULL
AND id != p_reservation_id
LIMIT 1;

IF existing_invoice_id IS NOT NULL THEN
UPDATE Invoice
SET total_amount = total_amount + slot_price,
tax_amount = tax_amount + (slot_price * 0.18)
WHERE id = existing_invoice_id;

UPDATE Reservation
SET invoice_id = existing_invoice_id
WHERE id = p_reservation_id;
ELSE
INSERT INTO Invoice (total_amount, tax_amount,
status, type)
VALUES (slot_price, slot_price * 0.18, 'unpaid',
'contract')
RETURNING id INTO existing_invoice_id;

UPDATE Reservation
SET invoice_id = existing_invoice_id
WHERE id = p_reservation_id;
END IF;
END;
$$;
```


3. **cancel_reservation_invoice** – ажурира фактура при откажување на резервација

```
CREATE OR REPLACE PROCEDURE cancel_reservation_invoice(
    p_reservation_id INT,
    p_invoice_id INT,
    p_contract_id INT
)
    LANGUAGE plpgsql AS $$
DECLARE
    slot_price NUMERIC(10,2);
    v_setup_id INT;
BEGIN
    IF p_invoice_id IS NULL THEN
        RETURN;
    END IF;

    IF p_contract_id IS NULL THEN
        UPDATE Invoice
        SET total_amount = 0,
            tax_amount = 0,
            status = 'refunded'
        WHERE id = p_invoice_id;
    ELSE
        SELECT setup_id INTO v_setup_id
        FROM Reservation WHERE id = p_reservation_id;

        SELECT price_per_slot INTO slot_price
        FROM WorkspaceSetup WHERE id = v_setup_id;

        UPDATE Invoice
        SET total_amount = GREATEST(total_amount -
slot_price, 0),
            tax_amount = GREATEST(tax_amount -
(slot_price * 0.18), 0)
        WHERE id = p_invoice_id;
    END IF;
END;
$;
```

Во Django апликацијата се имплементирани **сигнали** за автоматско повикување на PostgreSQL процедурите при креирање и ажурирање на резервации. На овој начин се овозможува синхронизација помеѓу апликацискиот слој и бизнис логиката имплементирана во базата.

Главната цел е автоматско генерирање на фактура при секоја нова резервација без дополнителна интервенција од корисникот. Кај резервации поврзани со активен договор, новите резервации автоматски се групираат во постоечка фактура, при што вкупниот износ се ажурира согласно новата резервација.

```

@receiver(post_save, sender=Reservation)
def handle_reservation_save(sender, instance, created,
**kwargs):
    if created:
        with connection.cursor() as cursor:
            if instance.contract_id is None:
                cursor.execute(
                    "CALL generate_single_invoice(%s, %s,
%s)",
                    [instance.id, instance.setup_id,
instance.responsible_member_id]
                )
            else:
                cursor.execute(
                    "CALL generate_contract_invoice(%s,
%s, %s)",
                    [instance.id, instance.setup_id,
instance.contract_id]
                )

@receiver(pre_save, sender=Reservation)
def handle_reservation_cancel(sender, instance,
**kwargs):
    if not instance.pk:
        return
    try:
        old = Reservation.objects.get(pk=instance.pk)
        if old.status != 'cancelled' and instance.status
== 'cancelled':
            with connection.cursor() as cursor:
                cursor.execute(
                    "CALL cancel_reservation_invoice(%s,
%s, %s)",
                    [instance.id, instance.invoice_id,
instance.contract_id]
                )
    except Reservation.DoesNotExist:
        pass

```

3.4 Иницијални тест податоци – seed_data.sql

За потребите на тестирање и демонстрација, во системот е внесен иницијален сет на тест податоци (seed data):

```

INSERT INTO Location (name, address, city) VALUES ('Center'...;

INSERT INTO Equipment (name, type) VALUES ('Dell Monitor 27"...;

INSERT INTO Member (first_name, last_name) VALUES ('Petar'...;

INSERT INTO MembershipPlan (type, price, discount_percentage) VALUES ('Basic'...;

INSERT INTO CoworkingCenter (name, contact_phone, email, location_id) VALUES ('Hub Skopje Center'...;

INSERT INTO Workspace (name, type, capacity, status, coworking_center_id) VALUES ('Room 101'...;

INSERT INTO WorkspaceSetup (workspace_id, version_number, price_per_slot, description) VALUES (1...;

INSERT INTO Contract (fixed_price, status, start_date, end_date, member_id) VALUES
( fixed_price 25000.00, status 'active', start_date '2026-01-01', end_date '2026-06-30', member_id 1),
( fixed_price 18000.00, status 'active', start_date '2026-02-01', end_date '2026-07-31', member_id 2),
( fixed_price 10000.00, status 'terminated', start_date '2025-09-01', end_date '2025-12-31', member_id 3),
( fixed_price 45000.00, status 'active', start_date '2026-04-01', end_date '2026-09-30', member_id 4),
( fixed_price 15000.00, status 'active', start_date '2026-03-01', end_date '2026-08-31', member_id 3);

INSERT INTO Reservation (date, slot, status, code, responsible_member_id, setup_id, invoice_id, contract_id) VALUES
( date '2026-05-25', slot 'morning', status 'confirmed', code 'RES-SKP-015', responsible_member_id 1, setup_id 1, invoice_id NULL, contract_id NULL),
( date '2026-05-27', slot 'afternoon', status 'pending', code 'RES-SKP-020', responsible_member_id 5, setup_id 1, invoice_id NULL, contract_id NULL),
( date '2026-06-01', slot 'morning', status 'confirmed', code 'RES-SKP-025', responsible_member_id 1, setup_id 1, invoice_id NULL, contract_id NULL),
( date '2026-06-01', slot 'afternoon', status 'confirmed', code 'RES-SKP-026', responsible_member_id 2, setup_id 1, invoice_id NULL, contract_id NULL),
( date '2026-06-02', slot 'morning', status 'confirmed', code 'RES-SKP-027', responsible_member_id 3, setup_id 1, invoice_id NULL, contract_id NULL);

```

4 SQL Views – Извештаи

Системот користи два SQL погледи (VIEW) за аналитички извештаи. Логиката е вградена во view-от и достапна преку едноставен SELECT.

4.1 Извештај 1 - Финансиски преглед на фактури

Цел: За потребите на финансиска анализа е креиран поглед (VIEW) кој овозможува месечен преглед на фактури според тип и статус. Извештајот прикажува вкупни платени, неплатени и рефундирани износи, број на фактури по статус, како и вкупен данок и генериран приход.

```

CREATE VIEW view_invoice_ledger AS
SELECT
    DATE_TRUNC('month', i.issue_date)::DATE AS month,
    i.type AS invoice_type,
    SUM(CASE WHEN i.status = 'paid' THEN
i.total_amount ELSE 0 END) AS total_paid,
    SUM(CASE WHEN i.status = 'unpaid' THEN
i.total_amount ELSE 0 END) AS total_unpaid,
    SUM(CASE WHEN i.status = 'refunded' THEN
i.total_amount ELSE 0 END) AS total_refunded,
    COUNT(CASE WHEN i.status = 'paid' THEN 1 END) AS
count_paid,
    COUNT(CASE WHEN i.status = 'unpaid' THEN 1 END) AS
count_unpaid,
    COUNT(CASE WHEN i.status = 'refunded' THEN 1 END) AS
count_refunded,
    SUM(i.total_amount) AS total_amount,
    SUM(i.tax_amount) AS total_tax

```

```
FROM Invoice i
GROUP BY DATE_TRUNC('month', i.issue_date), i.type
ORDER BY month DESC, i.type;

SELECT * FROM view_invoice_ledger;
```

Каде се користи во апликацијата: На GET /api/reports/ го извршува SELECT * FROM view_invoice_ledger и ги враќа податоците до React фронтендот. На ReportsPage се прикажува како табела „Invoice Status Ledger“ со реалните вредности од базата.

4.2 Извештај 2 – Пополнетост на центри

Цел: Овој извештај ги спојува CoworkingCenter, Workspace, WorkspaceSetup и Reservation за да овозможи прецизен мониторинг на искористеноста на физичкиот простор преку споредба на вкупниот капацитет на центрите со реално направените резервации на специфичен датум.

```
CREATE VIEW center_occupancy AS
SELECT
    cc.id AS center_id,
    cc.name AS center_name,
    l.city,
    r.date AS reservation_date,
    COUNT(r.id) AS total_bookings,
    SUM(w.capacity) AS total_capacity_used,
    (SELECT SUM(capacity) FROM Workspace WHERE
    coworking_center_id = cc.id) AS total_capacity
FROM CoworkingCenter cc
    JOIN Location l ON cc.location_id = l.id
    JOIN Workspace w ON w.coworking_center_id =
cc.id
    JOIN WorkspaceSetup ws ON ws.workspace_id = w.id
    JOIN Reservation r ON r.setup_id = ws.id
WHERE r.status = 'confirmed'
GROUP BY cc.id, l.city, r.date
ORDER BY r.date DESC, total_bookings DESC;

SELECT * FROM center_occupancy
WHERE reservation_date = CURRENT_DATE;
```

Каде се користи во апликацијата: Исто на GET /api/reports/ го извршува SELECT * FROM center_occupancy и ги враќа резултатите. На ReportsPage се прикажува како табела „Center Occupancy“ — со можност за филтрирање по CURRENT_DATE за преглед на денешната пополнетост.

5 Опис на апликацијата

CoWorkHub е database-first апликација наменета за управување со coworking центри, резервации, договори и фактурирање, имплементирана со PostgreSQL, Django REST Framework и React (TypeScript). Апликацијата е организирана во 3 сервиси кои комуницираат меѓусебно: PostgreSQL - база на податоци, Django backend – изложува REST API на порта 8000 и React Frontend – single page апликација на порта 5173.

Бизнис логиката е имплементирана во базата преку складирани процедури и views, додека Django signals овозможуваат нивно автоматско повикување при работа со резервации, како и административен интерфејс за CRUD на моделите.

5.1 Административен панел

Апликацијата користи Django Admin како главен интерфејс за управување со податоците. Преку административниот панел се овозможува CRUD функционалност за членови, претплати, резервации, договори, работни простори и конфигурации. Административниот панел е достапен на /admin/.

На сликата подолу е прикажана табелата за Резервации:

ID	CODE	DATE	SLOT	STATUS	RESPONSIBLE MEMBER	WORKSPACE	INVOICE	CONTRACT
16	RES-2A06DA99	May 15, 2026	Morning	Pending	Ana Angelovska	Open Desk A	Invoice #12	-
13	aaa	May 12, 2026	Evening	Pending	Mihail Georgiev	Conference Room Alpha	Invoice #9	-
30	RES-SKP-030	May 12, 2026	Morning	Cancelled	Ana Angelovska	Open Desk A	Invoice #6	Contract object (2)
9	RES-SKP-029	June 3, 2026	Morning	Confirmed	Martina Petkovska	Conference Room Alpha	Invoice #6	Contract object (2)
8	RES-SKP-028	June 2, 2026	Afternoon	Confirmed	Mihail Georgiev	Room 101	Invoice #7	Contract object (4)
7	RES-SKP-027	June 2, 2026	Morning	Confirmed	Petar Petrovski	Room 101	Invoice #5	Contract object (1)
6	RES-SKP-026	June 1, 2026	Afternoon	Confirmed	Martina Petkovska	Room 101	Invoice #6	Contract object (2)
5	RES-SKP-025	June 1, 2026	Morning	Confirmed	Petar Petrovski	Room 101	Invoice #5	Contract object (1)
4	RES-SKP-022	May 28, 2026	Afternoon	Cancelled	Mihail Georgiev	Conference Room Alpha	Invoice #4	-
3	RES-SKP-021	May 28, 2026	Morning	Confirmed	Ana Angelovska	Conference Room Alpha	Invoice #3	-
2	RES-SKP-020	May 27, 2026	Afternoon	Pending	Stefan Trajkovski	Open Desk A	Invoice #2	-
1	RES-SKP-015	May 25, 2026	Morning	Cancelled	Petar Petrovski	Open Desk A	Invoice #1	-

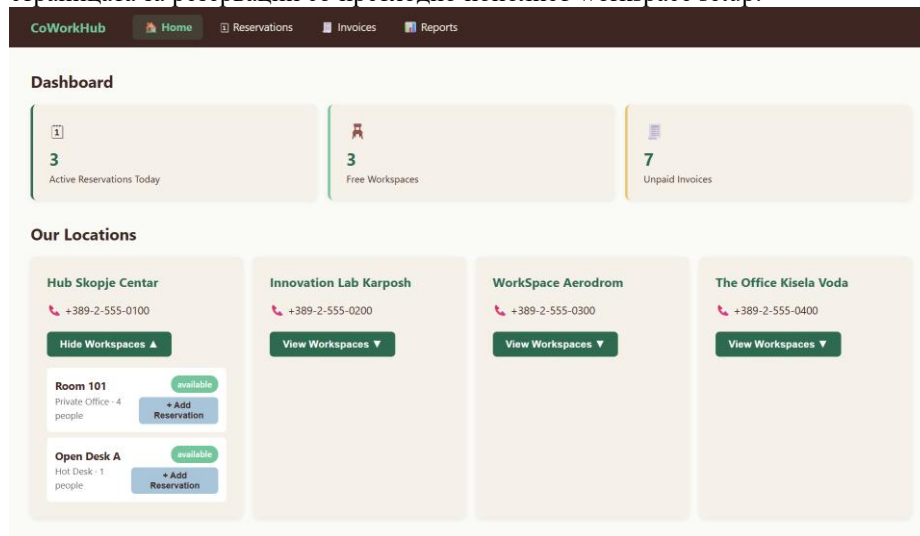
5.2 Dashboard (Почетна страна)

Почетната страница дава брз преглед на тековната состојба на системот преку 3 статистички картички кои се полнат во реално време со GET/api/stats/:

- Active Reservations Today – број на резервации со статус pending или confirmed за денес

- Free Workspaces – број на работни простори со статус available
- Unpaid Invoices – број на неплатени фактури

Под статистиките е прикажана листа на сите coworking центри (GET/api/centers/). За сите од нив може да се прошири листата на работни простори со нивен тип, капацитет и статус. За простори со статус available директно се нуди копче „+ Add Reservation“ кое го пренасочува корисникот на страницата за резервации со претходно пополнет workspace setup.



5.3 Страница за приказ на резервации

Страницата за резервации нуди два режими на приказ кои може да се менуваат:

- Табеларен приказ – ги прикажува сите резервации со: код, член, workspace, датум, термин, поврзан договор и фактура, и статус. Поддржано е филтрирање по статус (All / Pending / Confirmed / Cancelled). За секоја активна резервација постои копче за откажување кое повикува: PATCH/api/reservations/{id}/cancel/ - при тоа Django signal автоматски ја ажурира поврзаната фактура преку процедурата cancel_reservation_invoice.

CoWorkHub

Home

Reservations

Invoices

Reports

Reservations

Table

Calendar

+ New Reservation

All

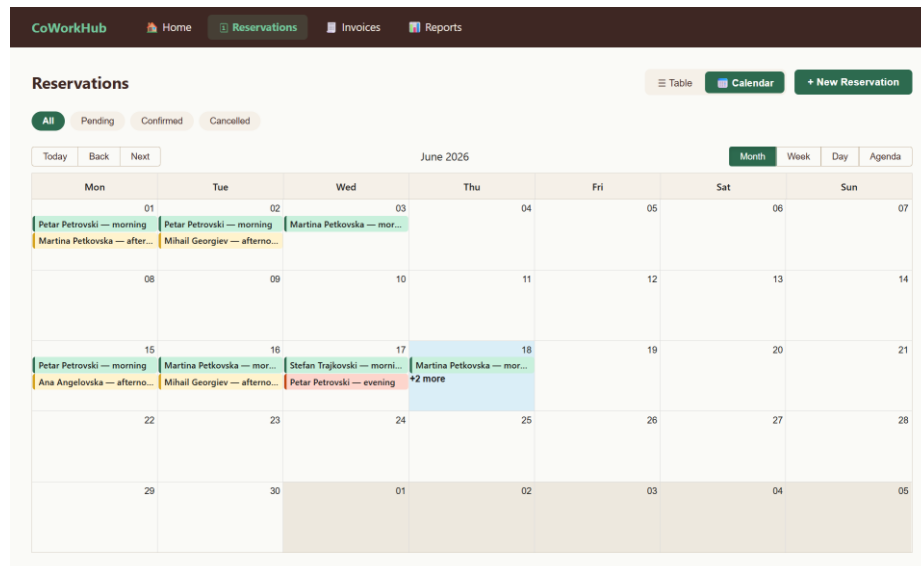
Pending

Confirmed

Cancelled

Code	Member	Workspace	Date	Slot	Contract	Invoice	Status	Action
RES-SKP-030	Petar Petrovski	—	2026-06-15	morning	#1	—	confirmed	Cancel
RES-SKP-031	Ana Angelovska	—	2026-06-15	afternoon	—	—	confirmed	Cancel
RES-SKP-032	Martina Petkovska	—	2026-06-16	morning	#2	—	confirmed	Cancel
RES-SKP-033	Mihail Georgiev	—	2026-06-16	afternoon	#4	—	confirmed	Cancel
RES-SKP-034	Stefan Trajkovski	—	2026-06-17	morning	—	—	confirmed	Cancel
RES-SKP-035	Petar Petrovski	—	2026-06-17	evening	#1	—	confirmed	Cancel
RES-SKP-036	Martina Petkovska	—	2026-06-18	morning	#2	—	confirmed	Cancel
RES-SKP-037	Ana Angelovska	—	2026-06-18	afternoon	—	—	confirmed	Cancel
RES-SKP-020	Stefan Trajkovski	—	2026-05-27	afternoon	—	#2	pending	Cancel
RES-SKP-021	Ana Angelovska	—	2026-05-28	morning	—	#3	confirmed	Cancel
RES-SKP-022	Mihail Georgiev	—	2026-05-28	afternoon	—	#4	confirmed	Cancel
RES-SKP-025	Petar Petrovski	—	2026-06-01	morning	#1	#5	confirmed	Cancel
RES-SKP-026	Martina Petkovska	—	2026-06-01	afternoon	#2	#5	confirmed	Cancel
RES-SKP-027	Petar Petrovski	—	2026-06-02	morning	#1	#5	confirmed	Cancel
RES-SKP-029	Martina Petkovska	—	2026-06-03	morning	#2	#5	confirmed	Cancel
RES-SKP-028	Mihail Georgiev	—	2026-06-02	afternoon	#4	#7	cancelled	
RES-SKP-038	Mihail Georgiev	—	2026-06-18	evening	#4	—	cancelled	
RES-SKP-015	Petar Petrovski	—	2026-05-25	morning	—	#1	cancelled	

- Календарски приказ — ги прикажува резервациите на месечен календар. Секој термин е означен со различна боја: утринскиот со зелена, попладневниот со жолта, вечерниот со црвена.



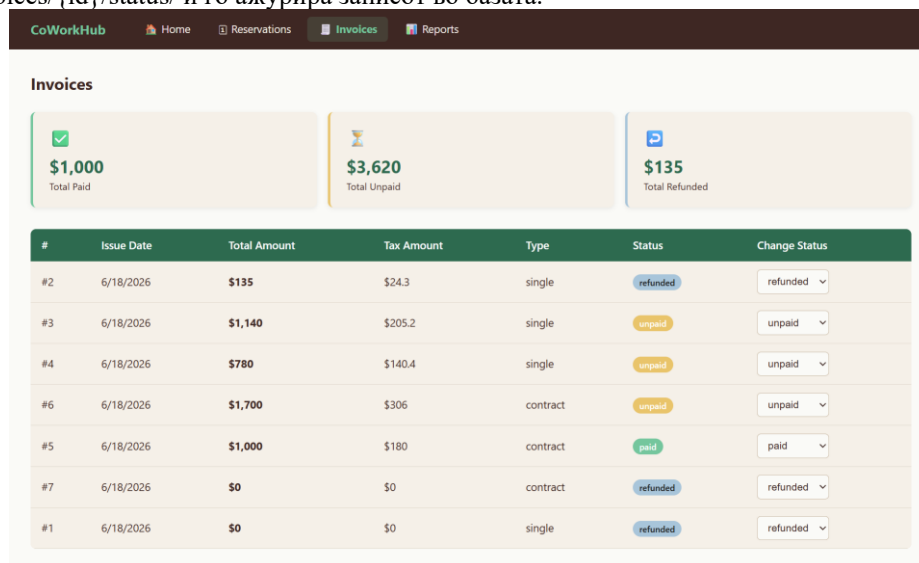
5.4 Форма за внес на резервации

Следниот екран ја покажува формата за внес на резервација, каде корисникот внесува датум, термин, workspace setup, одговорен член и дали резервацијата е дел од деловен договор или не. По зачувување на податоците, Django signal автоматски ја повикува соодветната процедура (`generate_single_invoice` или `generate_contract_invoice`) и се генерира или ажурира соодветна фактура.

5.5 Фактури

Страницата за фактури прикажува три суммарни картички со вкупниот износ по статус: платени, неплатени и рефундирани — пресметано директно на frontend од добиените податоци.

Под картичките е табела со сите фактури (GET /api/invoices/) со: број, датум на издавање, вкупен износ, данок, тип (single / contract) и статус. За секоја фактура постои dropdown за промена на статусот кој веднаш го повикува PATCH /api/invoices/{id}/status/ и го ажурира записот во базата.



5.6 Извештаи

На екранот подолу е прикажана посебна страница за приказ на аналитички податоци од системот. Страницата ги визуелизира резултатите од двата SQL views преку графици и табели:

- Invoice Status — Pie chart кој ги прикажува платените, неплатените и рефундираните фактури по удел во вкупниот износ. Под графикот е целосна табела со сите редови од view_invoice_ledger, групирани по месец и тип.
- Center Occupancy — Bar chart кој за секој центар прикажува две вредности: искористен капацитет и број на резервации. Поддржано е филтрирање помеѓу Today и This Week — при промена, се испраќа нов барање GET /api/reports/?period=week кое го прилагодува SQL филтерот на view-от.

CoWorkHub Home Reservations **Invoices** Reports

Invoices

\$1,000
Total Paid

\$3,620
Total Unpaid

\$135
Total Refunded

#	Issue Date	Total Amount	Tax Amount	Type	Status	Change Status
#2	6/18/2026	\$135	\$24.3	single	refunded	refunded
#3	6/18/2026	\$1,140	\$205.2	single	unpaid	unpaid
#4	6/18/2026	\$780	\$140.4	single	unpaid	unpaid
#6	6/18/2026	\$1,700	\$306	contract	unpaid	unpaid
#5	6/18/2026	\$1,000	\$180	contract	paid	paid
#7	6/18/2026	\$0	\$0	contract	refunded	refunded
#1	6/18/2026	\$0	\$0	single	refunded	refunded

(reports страна на frontend: <http://localhost:5173/reports>)

CoworkHub Reports Admin

Reports

Invoice Status Ledger									
Month	Invoice Type	Total Paid	Total Unpaid	Total Refunded	Count Paid	Count Unpaid	Count Refunded	Total Amount	Total Tax
May 1, 2026	contract	500.00	2700.00	0	1	2	0	3200.00	576.00
May 1, 2026	single	780.00	1275.00	0.00	1	2	1	2055.00	369.90

Center Occupancy						
Center Id	Center Name	City	Reservation Date	Total Bookings	Total Capacity Used	Total Capacity
2	Innovation Lab Karposh	Skopje	June 3, 2026	1	12	12
1	Hub Skopje Centar	Skopje	June 2, 2026	2	8	5
1	Hub Skopje Centar	Skopje	June 1, 2026	2	8	5
2	Innovation Lab Karposh	Skopje	May 28, 2026	2	24	12

(reports страна на backend: <http://localhost:8000/reports/>)

6 DevOps Инфраструктура (дополнително)

6.1 Докеризација на апликацијата (Dockerfiles)

Со цел да се обезбеди брзо, изолирано и конзистентно стартување на системот во различни средини, CoWorkHub е целосно софтверски контејнеризиран со користење на соодветни DevOps алатки.

Системот е поделен на микросервисна архитектура каде секоја компонента се изолира во сопствен Docker контејнер преку соодветен Dockerfile:

- Backend Сервис (Django)
- Frontend Сервис (React)

6.2 Мулти-контејнерско опкружување (Docker Compose)

Docker Compose - во коренот на репозиториумот е поставен `docker-compose.yml` фајл кој менаџира три меѓусебно поврзани сервиси:

- db Сервис: Ја стартува официјалната `postgres:15` слика.
- backend Сервис: Го стартува Django API-то
- frontend Сервис: Го подигнува спакуваниот React интерфејс на порта 80

6.3 Автоматизација и CI/CD pipeline (GitHub Actions)

За да се осигура континуирана интеграција (CI), во рамките на GitHub репозиториумот е конфигуриран автоматски workflow преку `.github/workflows/ci.yml`.

6.4 Оркестрација во Облак (Kubernetes – k8s/)

Како завршен чекор кон скалабилност, подготвена е папка `k8s/` која содржи детални декларативни манифести (YAML) кои опишуваат како CoWorkHub би се дистрибуирал во вистински кластер на cloud.

7 Заклучок

CoWorkHub претставува интегриран систем за управување со coworking простори, резервации и деловни договори. Со релационен модел, напредни SQL ограничувања и автоматизирани процедури и Django сигнали, системот обезбедува ефикасно управување со ресурси и финансии. Имплементацијата овозможува конзистентност на податоци и реална симулација на современи coworking бизнис процеси.

За тестирање на апликацијата доволно е во терминал да се внесе **docker compose up –build**. Оваа команда автоматски ќе ги изгради контејнерите за Django – backend и React – frontend, ќе ги извржи сите потребни миграции и ќе го стартува целиот систем. Откако процесот ќе заврши, апликацијата е веднаш достапна на следните локални адреси преку кој било веб-прелистувач:

- Кориснички интерфејс (Frontend): **`http://localhost:5173`**
- Бекенд API и Django Admin: **`http://localhost:8000`**

References

1. <https://www.postgresql.org/docs/current/>
2. <https://www.postgresql.org/docs/current/functions.html>
3. <https://www.psycopg.org/install/>
4. <https://docs.djangoproject.com/en/6.0/ref/databases/#postgresql-notes>